ChainGuard:
Smart Contract Security Auditor

Project Group 2

Abel S Mathew	CHN23CS004
Abraham Santhosh	CHN23CS013
Diya Pauly	CHN23CS073
Megha Ajith	CHN23CS128

Under the Guidance of

Ms. Alka Vijay

CSD415 Project Phase 1



Dept. of Computer Engineering
College of Engineering, Chengannur
APJ Abdul Kalam Technological University



Outline

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

1. Introduction
2. Problem statement
3. Literature review and research gaps
4. Proposed solution
5. System design: architecture, use case, data flow diagrams
6. Methodology
7. Requirements: functional, non-functional, hardware, software
8. Result
9. Conclusion and future scope
10. References

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

What is ChainGuard?

A command-line tool that audits Solidity smart contracts by running Slither first and then having three LLM agents (Prosecutor, Defender, Judge) review each warning.

- **High-Stakes Security:**Smart contract vulnerabilities can lead to irreversible financial losses after deployment.
- **Static tools:** Slither is fast, but many warnings are not real bugs, and it misses flaws in business logic.
- **LLMs:** They read code and explain problems, but sometimes invent bugs.
- **Our Approach:** ChainGuard uses Prosecutor, Defender, and Judge agents to examine findings, challenge potential false positives, and produce a final verdict.

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

To develop an intelligent smart contract security auditing system that detects and verifies genuine vulnerabilities, reduces false positives, and provides reliable explanations to improve security assessment before deployment.



Literature Review

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Paper Title (Year)	Methodology	Evaluation metrices	Gaps identified
Slither: A Static Analysis Framework For Smart Contracts - Feist et al (2019)	**S**tatic analysis framework for Solidity with built-in bug detectors	Parsing Speed 1s per contract and coverage 99.9%, 10.9% FP on reentrancy	Fixed rules; many false alarms; misses business-logic bugs
Large Language Model-Powered Smart Contract Vulnerability Detection:New Perspective - Hu et al. (2023)	An LLM reads contract code and reports problems	Uses wider code context; gives written explanations	Can invent bugs; inconsistent; prompt-sensitive
GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis - Sun et al. (2024)	GPT-3.5 with static program analysis (ANTLR AST parsing + static confirmation modules for data flow, order checks, and value comparisons).	Precision over 90% on token contracts, 57.14% on Web3Bugs; recall over 70%	One LLM pass; depends on static analysis coverage



Literature Review

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Authors (Year)	Methodology	Evaluvation metrices	Gaps Identified
Improving Factuality and Reasoning in Language Models through Multi-Agent Debate - Du et al. (2023)	LLM agents answer, read each other's answers, and revise over rounds	Arithmetic Acc: 81.8% GSM8K Math Acc: 85.0%	Higher API latency and cost; not tested on smart contracts.
ChatEval: Towards Better LLM-Based Evaluators Through Multi-Agent Debate - Chan et al. (2023)	Multi-agent LLM debate with different roles and communication strategies.	Accuracy: 63.8Cohen's Kappa: 0.40 Spearman Correlation: 0.684 Kendall-Tau: 0.632	Homogeneous LLMs; limited benchmarks; more rounds may reduce performance
Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems - Wei et al. (2025)	Multi-agent LLM auditing with BA, TA, and Hybrid modes	98% accuracy, 99% precision, 98% recall, 98.5% F1; Real-world: 47.6% (TA), 14.4% (BA) accuracy.	High cost; hard to coordinate; agent claims need checking

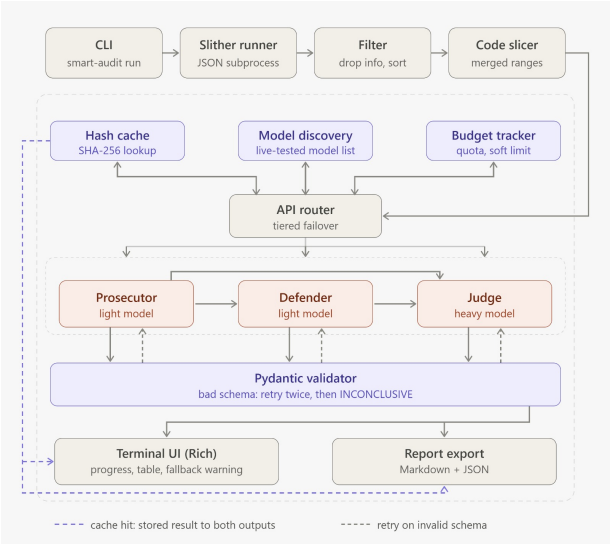
Proposed Solution

Vision: A command-line tool that runs Slither first, then asks three LLM agents to review each warning. The **Prosecutor** argues it is a real bug, the **Defender** argues it is not, and the **Judge** decides. The result keeps the verdict, the reasons and a suggested fix.

1. **Slither as the bug finder:** Slither finds candidate bugs, so the LLM does not search the whole contract.
2. **Filter and code slicer:** Cut the text sent to the LLM.
3. **Courtroom review:** Each warning is argued from both sides before a verdict is given.
4. **Free-tier resilience:** A cache, quota counters, live model discovery and provider fallback keep the tool working on free API tiers.

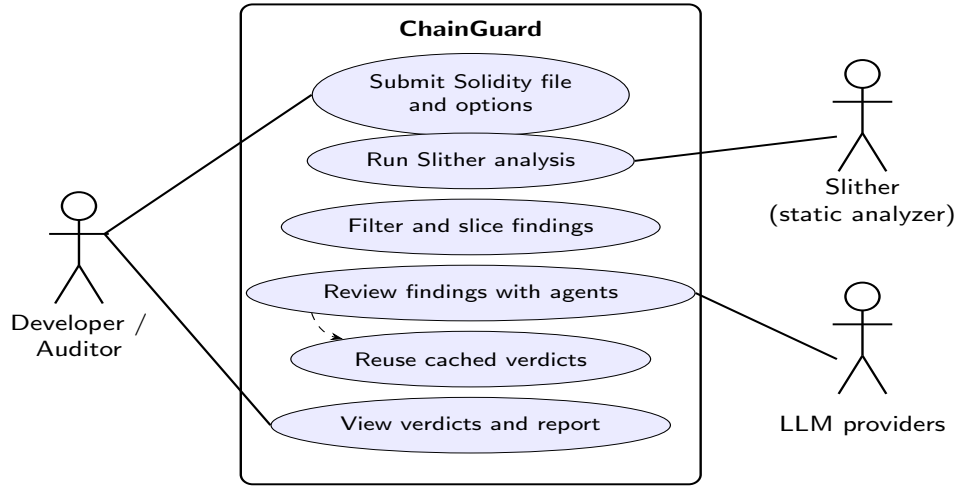
System Architecture

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References



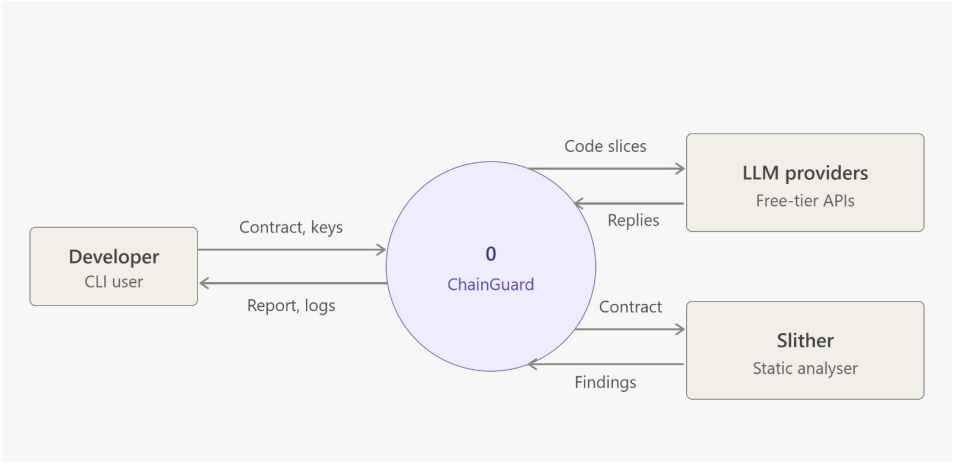
Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Use Case Diagram



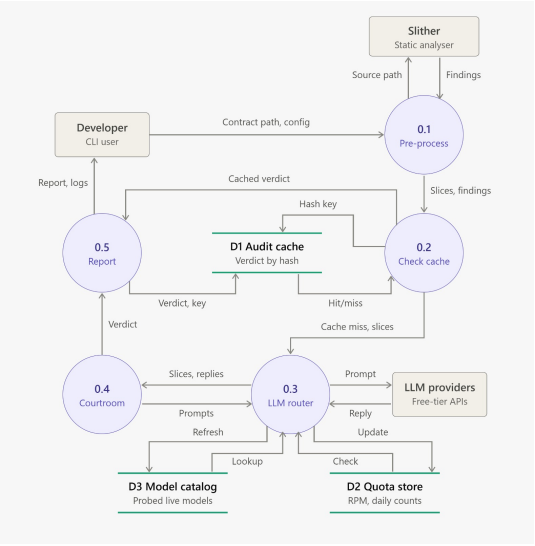
DFD Level 0 (Context Diagram)

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References



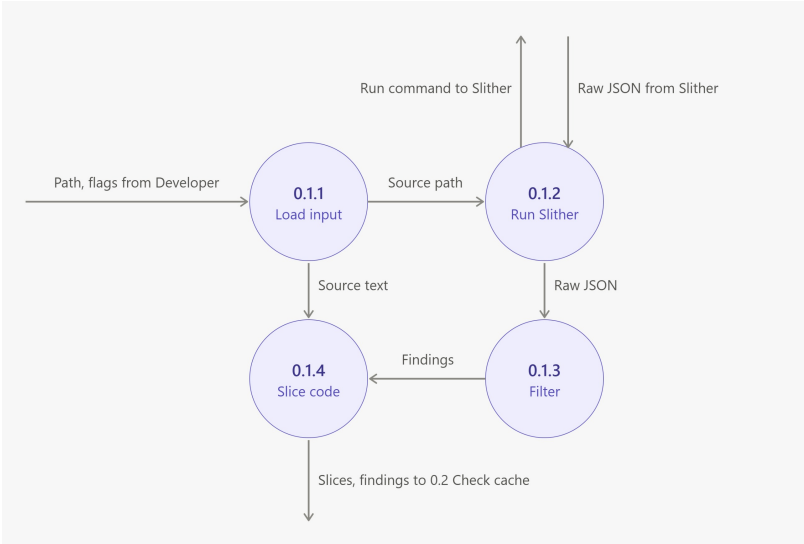
DFD Level 1

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References



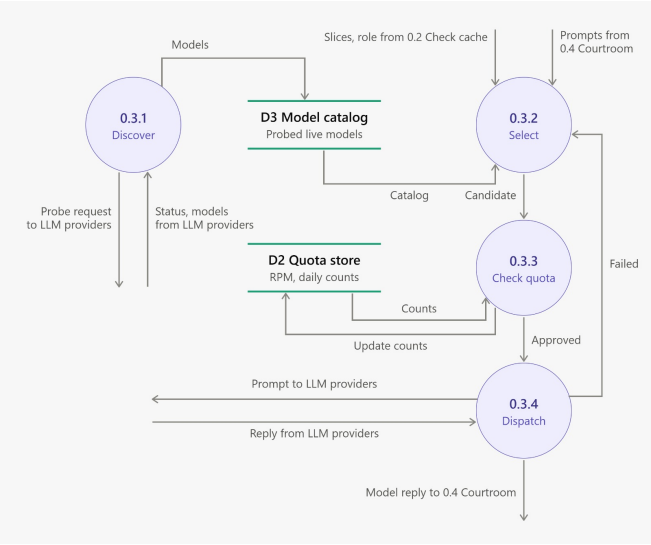
Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

DFD Level 2: Process 0.1 Pre-process



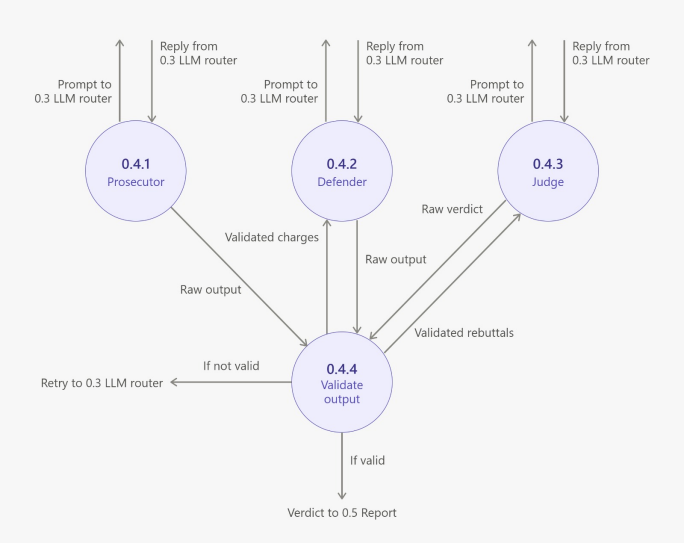
DFD Level 2: Process 0.3 LLM Router

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References



DFD Level 2: Process 0.4 Courtroom

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References



Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Phase 1: Pre-processing

- **Load input:** `smart-audit run <file>.sol`. The path can be a bare name, relative or absolute (bare names are looked up in `contracts/`). The file must end in `.sol`; API keys are read from `.env`.
- **Run Slither:** runs Slither as a local subprocess in JSON mode and reads the result in memory.
- **Filter:** drops Informational and Optimization findings, sorts the rest by impact and confidence, and can cap the count with `--max-findings`.
- **Slice code:** finds the functions behind each finding, merges overlapping line ranges, and records an error if slicing fails.

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Phase 2: Router, Cache and Checks

- **Hash cache:** SHA-256 of contract source and finding ID; repeated runs reuse the stored verdict.
- **Model discovery:** tests each candidate model with a real call, keeps the working list for 24 hours, and reuses the old list if the provider is unreachable.
- **Tier-based selection:**
  - Heavy tier (Judge): NVIDIA, then Groq, then Gemini.
  - Light tier (Prosecutor and Defender): Groq, then Gemini, then NVIDIA.
- **Quota check:** SQLite counters per provider and model. A hard check blocks any call over the limit; a soft limit of 0.85 switches provider before an HTTP 429 error.
- **Dispatch:** sends the prompt with a time-out; on failure the next candidate is tried. The answer is marked `degraded` if a fallback replied.

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Phase 3: Courtroom Agents

- **Schemas:** `ProsecutorCharge`, `DefenderResponse`, `JudgeVerdict` and `DebateRecord`.
- **Prosecutor:** states the suspected bug, its severity and an attack scenario, using the sliced code.
- **Defender:** points to guards, modifiers and state that make the code safe, and says whether it agrees.
- **Judge:** gives `CONFIRMED`, `FALSE_POSITIVE` or `INCONCLUSIVE`, with severity, confidence from 0 to 1, reasons and a fix.
- **Validate output (0.4.4):** schema check on every reply; bad replies are retried up to twice, then marked `INCONCLUSIVE`.
- **Orchestrator:** checks the cache, builds the code context, runs the three agents in order, notes any fallback, and saves the result. It stops cleanly if every provider is out of quota.

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Phases 4 and 5: Report and Evaluation

- **CLI entry point:** smart-audit run file .sol (Typer) runs the whole tool.
  Flags: –max-findings, –fast.
- **Pipeline wiring:** Connects Phase 1 (load, Slither, filter, slice), Phase 2 (cache, router, quota) and Phase 3 (Prosecutor, Defender, Judge) into one run over all findings.
- **Terminal UI (Rich):** Live progress, streamed agent steps, cache-hit marks, and a per-finding summary table.
- **Degraded-mode warning:** Shown when a fallback model replied, so results get manually verified.
- **Report export:** Writes a JSON report to the local directory, with bug location, rule broken, code, verdict, confidence and suggested fix.

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Functional Requirements

- **Solidity File Validation:** Accept Solidity file paths, resolve bare filenames from the contracts/ directory, and reject missing or invalid files.
- **Static Analysis Engine:** Execute Slither on the specified contract and retrieve vulnerability findings in JSON format.
- **Finding Prioritization:** Filter, rank, and limit findings based on impact and confidence, while extracting relevant code context.
- **Multi-Agent Validation:** Run the Prosecutor, Defender, and Judge agents sequentially to assess each potential vulnerability and produce a structured verdict.
- **Provider and Quota Management:** Select LLM providers based on availability, tier, and quota limits, with fallback support when necessary.
- **Report Generation:** Display verdicts, severity, confidence, explanations, and suggested fixes in the terminal, with JSON export support.



Non-Functional Requirements

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

- **Efficiency:** send as little text as possible to the LLM, through filtering, slicing and caching.
- **Reliability:** a bad model reply or an exhausted provider must not crash a run.
- **Usability:** one command to run an audit, with readable terminal output.
- **Explainability:** every verdict comes with reasons, a confidence value and, when confirmed, a fix.
- **Security and privacy:** API keys stay in a local `.env` file; temporary Slither output is deleted; only code slices are sent to LLM providers.
- **Cost:** work within free-tier API limits.
- **Maintainability:** separate modules for pre-processing, routing, agents and output, with typed schemas.
- **Portability:** runs wherever Python 3.10+ and Slither run.

Hardware Requirements

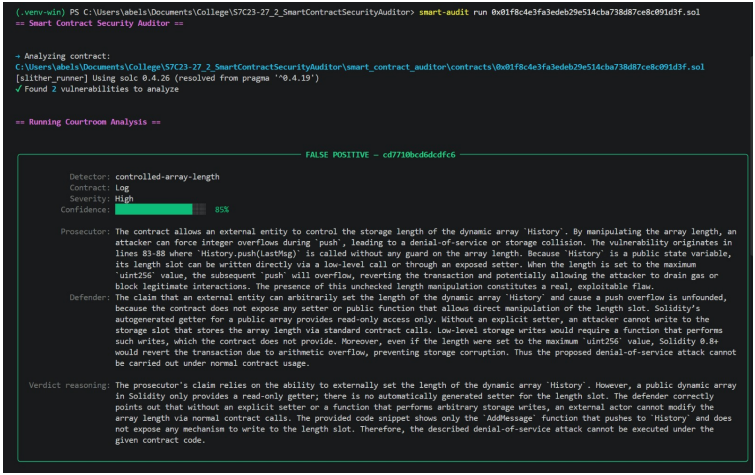
Item	Minimum	Recommended
Processor	Dual-core 64-bit CPU	Quad-core CPU (Core i5 or Ryzen 5 class)
RAM	4 GB	8 GB or more
Storage	2 GB free	5 GB free, SSD
Network	Internet connection	Stable broadband

Software Requirements

Category	Technologies
Platform	Windows, Linux or macOS; Python 3.10+; Terminal and Git
Static analysis	slither-analyzer; Solidity compiler (`solc`) managed by py-solc-x
LLM APIs	NVIDIA, Groq and Gemini (free-tier keys stored in `.env`)
Storage	SQLite (comes with Python) for the audit cache, quota store and model catalog
Python packages	Typer (command line), Rich (terminal output), Pydantic (schemas), python-dotenv (API keys), requests (API calls)
Dev tools	pytest, black, ruff for testing and code style

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Results



Results

CONFIRMED — 8bc543e1578f279e

Detector: reentrancy-eth
Contract: PrivateDeposit
Severity: High
Confidence: 95%

Prosecutor: The CashOut function performs an external call to msg.sender before updating the caller's balance, allowing a re-entrancy attack to drain funds from the PrivateDeposit contract.

Defender: The CashOut function lacks the Checks-Effects-Interactions pattern, allowing a malicious caller to re-enter and withdraw the same balance repeatedly. No other modifiers, mutexes, or balance adjustments precede the external call, and the fallback is payable, so a re-entrancy attack can drain the contract. The vulnerability is real and exploitable.

Verdict reasoning: The CashOut function performs an external low-level call (msg.sender.call.value(_am)()) before updating the caller's balance (balances -= _am). The contract also exposes a payable fallback function, allowing a malicious contract to re-enter CashOut during the external call and withdraw the same amount repeatedly, draining the contract's Ether. No mitigations such as checks-effects-interactions ordering or a reentrancy guard are present. This matches the classic DAO-style reentrancy vulnerability and is exploitable as described.

Patch: Reorder the function to follow checks-effects-interactions: first verify the amount, then deduct the balance, then make the external call. Alternatively, add a nonReentrant modifier (e.g., using OpenZeppelin's ReentrancyGuard) to protect the function. Example patch:

```
function CashOut(uint _am) public {
    require(_am <= balances, "Insufficient balance");
    balances -= _am;
    (bool success, ) = msg.sender.call{value: _am}("");
    require(success, "Transfer failed");
    TransferLog.AddMessage(msg.sender, _am, "CashOut");
}
```

Audit Summary

Finding ID	Detector	Verdict	Confidence	Degraded
cd7710bcd6dcdfc6	The contract allows an external entity t…	FALSE POSITIVE	85%	
8bc543e1578f279e	The CashOut function performs an externa…	CONFIRMED	95%	

1 confirmed · 1 false positive · 0 inconclusive (2 total)

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Conclusion

- **Review of every warning:** ChainGuard runs Slither, then has a Prosecutor, Defender and Judge review each warning.
- **Usable on free tiers:** Filtering, code slicing, caching and provider fallback keep it working within free API limits.
- **Robust to bad replies:** Schema checks and limited retries stop bad model replies from crashing a run.
- **Combined strengths:** The survey found that no single method solves contract auditing. Static analysis gives evidence, ML finds patterns, LLMs add context, and multiple agents add a second check.
- **Aid, not guarantee:** The tool helps human auditors. It does not guarantee a contract is secure.

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

Future Scope

1. **Evaluation:** Test on labelled and unseen contracts; measure precision, recall, F1-score, and false-positive rate.
2. **Preprocessing Packaging:** Add a preprocessing-only mode and package ChainGuard as a standalone executable.
3. **RAG Integration:**Use a local vulnerability knowledge base to improve analysis and reduce false positives.
4. **Multi-Contract Support:** Analyse multiple contracts, imports, dependencies, and cross-contract vulnerabilities.
5. **CI/CD integration:**Automate audits on every commit or pull request.
6. **Web Dashboard:** Provide an interface to monitor audits, view vulnerabilities, and download reports.

Introduction
Problem Statement
Literature Review
Proposed Solution
System Architecture
System Architecture
Use Case Diagram
DFD Level 0
DFD Level 1
DFD Level 2
Methodology
Requirements
Results
Conclusion
Future Scope
References

References I

1. J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. WETSEB*, 2019.
2. S. Hu *et al.*, "Large language model-powered smart contract vulnerability detection: New perspectives," arXiv:2310.01152, 2023.
3. Y. Sun *et al.*, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. ICSE*, 2024.
4. Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," arXiv:2305.14325, 2023.
5. C.-M. Chan *et al.*, "ChatEval: Towards better LLM-based evaluators through multi-agent debate," arXiv:2308.07201, 2023.
6. Z. Wei *et al.*, "Advanced smart contract vulnerability detection via LLM-powered multi-agent systems," *IEEE Trans. Softw. Eng.*, vol. 51, no. 10, pp. 2830–2846, 2025.

Thank You